

TypeScript et la Programmation Orientée Objet



Qu'est ce que TypeScript

Créé en 2012 par Microsoft

A pour but de répondre aux limitations structurelles de JavaScript dans les projets de grande envergure

Est un sur-ensemble de JavaScript qui ajoute un système de types statiques facultatifs

Limitations de JavaScript

JS



```
1  function addition(a, b) {  
2      return a + b;  
3  }  
4  
5  console.log(addition(2, 3));      // 5  
6  console.log(addition("2", 3));   // "23"  
7  console.log(addition(true, {})); // "true[object Object]"
```

Limitations de JavaScript



```
1 function addition(a: number, b: number): number {
2     return a + b;
3 }
4
5 console.log(addition(2, 3));           // OK
6 console.log(addition("2", 3));        // ✗ Erreur
7 console.log(addition(true, {}));      // ✗ Erreur
8
9 // Argument of type 'string' is not assignable to parameter of type 'number'.
10 // Argument of type 'boolean' is not assignable to parameter of type 'number'.
11 // Argument of type '{}' is not assignable to parameter of type 'number'.
```

Limitations de JavaScript



```
1  const user = {  
2      name: "Alice",  
3      age: 30  
4  };  
5  
6  console.log(user.email);    // undefined  
7  user.email.toLowerCase();  // TypeError: Cannot read property 'toLowerCase' of undefined
```

Limitations de JavaScript



```
1  type User = {
2      name: string;
3      age: number;
4  };
5
6  const user: User = {
7      name: "Alice",
8      age: 30
9  };
10
11 console.log(user.email);
12 console.log(user.email.toLowerCase());
13
14 // La propriété 'email' n'existe pas sur le type 'User'.
15 // Ne compilera pas, donc impossible de lever une erreur sur 'user.email.toLowerCase()'
```

Langage interprété / compilé



compilateur



interpréteur



Les avantages de TypeScript : le typage



```
1  const isDone: boolean = true;
2  const age: number = 30;
3  const username: string = "Nicolas";
4  const numbers: number[] = [1, 2, 3];
5  const nothing: null = null;
6  const notDefined: undefined = undefined;
7  const object: Object = {"key": "value"};
```


Les avantages de TypeScript : ciblage des versions



```
1  {  
2    "compilerOptions": {  
3      "target": "ES2022",  
4      //"target": "ES2017",  
5      //"target": "ESNext"  
6    }  
7  }
```

Permet à partir d'une base de code uniforme, de compiler vers différentes versions de JavaScript.

Rendu à la compilation

TypeScript n'existe plus au runtime, les types sont effacés.

Le typage est donc un filet de sécurité à la compilation, pas une garantie d'exécution.



```
1  type User = {  
2      name: string;  
3      age: number;  
4  };  
5  
6  const user: User = {  
7      name: "Alice",  
8      age: 30  
9  };
```



```
1  "use strict";  
2  const user = {  
3      name: "Alice",  
4      age: 30  
5  };
```

Quand utiliser TypeScript / JavaScript

TypeScript :

- projets de grande envergure ou avec un back-end conséquent
- maintenance à long terme
- pour les projets avec de nombreux collaborateurs

JavaScript :

- petites fonctionnalités
- dans des environnement 100% sous JavaScript
- projets à courte durée de vie

La programmation orientée objet

La POO (Programmation Orientée Objet) organise le code en objets qui combinent données (attributs) et comportements (méthodes).

Ces objets sont instanciés via des classes.

La PF (Programmation Fonctionnelle) organise le code autour de fonctions pures, de la composition et de la transformation de données.



```
1  function deposit(balance: number, amount: number): number {  
2      return balance + amount;  
3  }  
4  
5  function deposit(balance: number, amount: number): number {  
6      return balance - amount;  
7  }  
8  
9  let balance: number = 100;  
10 balance = deposit(balance, 50);  
11 console.log(balance);
```



```
1  class BankAccount {  
2      private balance: number;  
3  
4      constructor(initialBalance: number) {  
5          this.balance = initialBalance;  
6      }  
7  
8      public deposit(amount: number): void {  
9          this.balance += amount;  
10     }  
11  
12     public withdraw(amount: number): void {  
13         this.balance -= amount;  
14     }  
15  
16     public getBalance(): number {  
17         return this.balance;  
18     }  
19 }  
20  
21 const account = new BankAccount(100);  
22 account.deposit(50);  
23 console.log(account.getBalance());
```



```
1  function deposit(balance: number, amount: number): number {
2      return balance + amount;
3  }
4
5  function withdraw(balance: number, amount: number): number {
6      return balance - amount;
7  }
8
9  let name_1: string = "Bob";
10 let balance_1: number = 100;
11 let currency_1: string = "€";
12
13 let name_2: string = "Alice";
14 let balance_2: number = 250;
15 let currency_2: string = "$";
```



```
1  class BankAccount {
2      private name: string;
3      private balance: number;
4      private currency: string;
5
6      constructor(name: string, initialBalance: number, currency: string) {
7          this.name = name;
8          this.balance = initialBalance;
9          this.currency = currency;
10     }
11
12     public deposit(amount: number): void {
13         this.balance += amount;
14     }
15
16     public withdraw(amount: number): void {
17         this.balance -= amount;
18     }
19
20     public getBalance(): number {
21         return this.balance;
22     }
23 }
24
25 const account_1 = new BankAccount("Bob", 100, "€");
26 const account_2 = new BankAccount("Alice", 250, "$");
```


Représentation de la classe BankAccount

```
1 class BankAccount {
2     private name: string;
3     private balance: number;
4     private currency: string;
5
6     constructor(name: string, initialBalance: number, currecy: string) {
7         this.name = name;
8         this.balance = initialBalance;
9         this.currency = currecy;
10    }
11
12    public deposit(amount: number): void {
13        this.balance += amount;
14    }
15
16    public withdraw(amount: number): void {
17        this.balance -= amount;
18    }
19
20    public getBalance(): number {
21        return this.balance;
22    }
23 }
24
25 const account_1 = new BankAccount("Bob", 100, "€");
26 const account_2 = new BankAccount("Alice", 250, "$");
```

```
[LOG]: BankAccount: {
    "name": "Bob",
    "balance": 100,
    "currency": "€"
}
```

Décortilage de la classe BankAccount



```
1 private name: string;  
2 private balance: number;  
3 private currency: string;
```

Attributs : données ou propriétés associées à un objet qui décrivent l'état ou les caractéristiques de celui-ci.

Décorticage de la classe BankAccount

```
1  constructor(name: string, initialBalance: number, currency: string) {  
2      this.name = name;  
3      this.balance = initialBalance;  
4      this.currency = currency;  
5  }  
6  
7  public deposit(amount: number): void {  
8      this.balance += amount;  
9  }  
10  
11 public withdraw(amount: number): void {  
12     this.balance -= amount;  
13 }  
14  
15 public getBalance(): number {  
16     return this.balance;  
17 }
```

Méthodes : fonctions définies dans la classe, qui décrivent les actions que l'objet peut effectuer, elles manipulent les attributs ou effectuent des opérations liées à l'objet.

Les modificateurs d'accès en POO

public (par défaut) : accessible partout

private : accessible uniquement dans la classe

protected : accessible dans la classe et ses sous-classes

Les modificateurs d'accès en POO



```
1  class BankAccount {  
2      public balance: number;  
3  
4      constructor(initialBalance: number) {  
5          this.balance = initialBalance;  
6      }  
7  }  
8  
9  const account = new BankAccount(100);  
10 account.balance = 150;  
11 console.log(account.balance);
```

Les modificateurs d'accès en POO

```
1  class BankAccount {  
2      private balance: number;  
3  
4      constructor(initialBalance: number) {  
5          this.balance = initialBalance;  
6      }  
7  }  
8  
9  const account = new BankAccount(100);  
10 account.balance = 150;  
11 console.log(account.balance);
```

❌ *Property 'balance' is private and only accessible within class 'BankAccount'!*

Les modificateurs d'accès en POO

```
1 class BankAccount {
2     private balance: number;
3
4     constructor(initialBalance: number) {
5         this.balance = initialBalance;
6     }
7
8     public setBalance(balance: number): void {
9         this.balance = balance;
10    }
11
12    public getBalance(): number {
13        return this.balance;
14    }
15 }
16
17 const account = new BankAccount(100);
18 account.setBalance(150);
19 console.log(account.getBalance());
```

✓ '150'

Les attributs et méthodes statiques

```
1 class Account {  
2     static interestRate: number = 0.05;  
3     public balance: number;  
4  
5     public constructor(balance: number) {  
6         this.balance = balance;  
7     }  
8 }  
9  
10 console.log(Account.interestRate); // ✓  
11 const a1 = new Account(1000);  
12 console.log(a1.balance); // ✗
```

Les attributs statiques appartiennent à la classe dans sa globalité, pas à une seule instance.

Si on change la valeur de l'attribut statique, sa valeur change pour toutes les instances

Les attributs et méthodes statiques

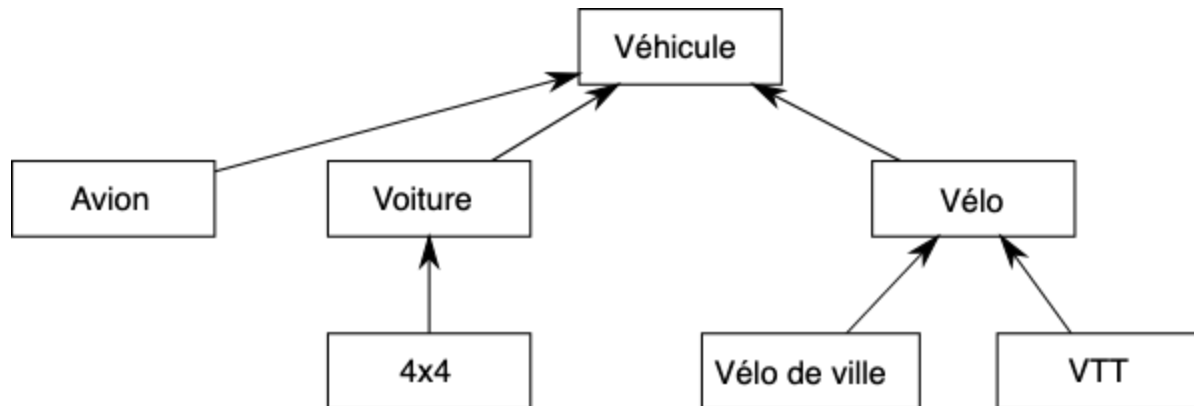


```
1  class User {  
2      static createFullName(firstName: string, lastName: string): string {  
3          return `${firstName} ${lastName}`;  
4      }  
5  }  
6  
7  console.log(User.createFullName("John", "Smith")); // ✓
```

Dans une méthode d'instance : *this* = l'instance

Dans une méthode statique : *this* = la classe

L'héritage



L'héritage

```
1  class Animal {
2      protected name: string;
3
4      public constructor(name: string) {
5          this.name = name;
6      }
7
8      public move(): void {
9          console.log(`${this.name} is moving`);
10     }
11 }
12
13
14 class Rabbit extends Animal {
15     private earLength: number;
16
17     public constructor(name: string, earLength: number) {
18         super(name);
19         this.earLength = earLength;
20     }
21
22     public hop(): void {
23         console.log(`${this.name} with ears ${this.earLength}cm long is hopping`);
24     }
25 }
26
27 const myRabbit = new Rabbit("Bunny", 12);
28 myRabbit.move(); // 'Bunny is moving'
29 myRabbit.hop(); // 'Bunny with ears 12cm long is hopping'
```

L'héritage est un mécanisme de la programmation orientée objet qui permet à une classe de réutiliser et étendre les propriétés et méthodes d'une autre classe (classe parente).

Une classe ne peut hériter que d'une seule autre classe.

Surcharge de méthodes dans une classe fille

```
1 class Animal {
2     public speak(): void {
3         console.log("An animal is making noise.");
4     }
5 }
6
7 class Dog extends Animal {
8     public override speak(): void {
9         console.log("The dog barks.");
10    }
11 }
12
13 const d = new Dog();
14 d.speak(); // "The dog barks."
```

La classe Dog surcharge la méthode speak de sa classe mère, elle prend donc le pas sur cette dernière.

On lui donne l'attribut 'override' pour indiquer que c'est une méthode qui en surcharge une autre.

Les classes abstraites

```
1  abstract class Animal {
2      public constructor(protected name: string) {}
3
4      public move(): void {
5          console.log(`${this.name} is moving`);
6      }
7
8      abstract makeSound(): void;
9  }
10
11
12  class Rabbit extends Animal {
13      public makeSound(): void {
14          console.log(`${this.name} squeaks!`);
15      }
16
17      public hop(): void {
18          console.log(`${this.name} is hopping`);
19      }
20  }
```

Une classe abstraite est une classe qui ne peut pas être instanciée directement, mais qui sert de modèle pour d'autres classes qui en hériteront.

Une classe abstraite peut contenir des méthodes abstraites (déclarées mais sans implémentation).

Les interfaces

```
1 interface Animal {
2     name: string;
3     move(): void;
4 }
5
6 class Rabbit implements Animal {
7     public readonly name: string;
8
9     public constructor(name: string) {
10         this.name = name;
11     }
12
13     public move(): void {
14         console.log(`${this.name} is hopping`);
15     }
16
17     public hop(): void {
18         console.log(`${this.name} is jumping!`);
19     }
20 }
21
```

Une interface est un contrat qui définit la forme d'un objet : quelles propriétés et quelles méthodes une classe doit posséder.

Une classe peut implémenter autant d'interfaces qu'elle le souhaite.